

Sveučilište u Zagrebu
Prirodoslovno-matematički fakultet

Distribuirani procesi

**Pouzdana difuzija (broadcast) u
asinkronim sustavima uz
mogućnost prekida rada procesa**

Ivana Rizner, Jelena Žaja, Nikola Mamić

Zagreb, Svibanj 2026

Sadržaj

1	Uvod	1
2	Distribuirani sustavi i komunikacijske apstrakcije	2
2.1	Distribuirani sustavi	2
2.2	Asinkrono okruženje	2
2.3	Crash failure model	3
2.4	Problem broadcast komunikacije	3
2.5	Komunikacijske apstrakcije broadcasta	4
2.6	Model sustava CAMP	5
3	Uniformna pouzdana difuzija (URB)	6
3.1	Svojstva URB-a	6
3.2	Uniformnost	6
3.3	Važnost uniformnosti	7
3.4	Primjer	7
3.5	URB paradigma	8
4	URB algoritam	9
4.1	Strukture podataka	10
4.2	Pseudokod algoritma	10
4.3	Korektnost algoritma	10
4.4	ACK potvrde i uvjet za isporuku (deliver)	11
5	Proširenja algoritma	12
5.1	FIFO	13

5.1.1	Primjer	13
5.1.2	Jednostavna konstrukcija	14
5.1.3	Teorem	15
5.2	CO-URB	16
5.2.1	Parcijalni poredak nad porukama	16
5.2.2	Kauzalna isporuka poruka	17
5.2.3	Odnos FIFO i CO redoslijeda	17
5.2.4	Jednostavna konstrukcija CO-broadcasta iz URB-broadcasta	20
5.2.5	Konstrukcija CO-broadcasta iz FIFO-broadcasta	21
5.2.6	Konstrukcija CO-broadcasta iz URB-broadcasta pomoću vektora	28
6	Zaključak	39
7	Analiza i ograničenja algoritma	39

1 Uvod

Jedan od izazova pri dizajniranju asinkronih distribuiranih sustava je razvoj robusnog sustava razmjena poruka, pogotovo u slučajevima kada može doći do neočekivanog prestanka rada (*crash-a*) jednog ili više procesa. U ovom radu bavimo se algoritmima za difuziju (*broadcast*) u takvim sustavima. Naime, uz neočekivani prestanak rada nekih od procesa u sustavu, semantiku difuzije da se poruka šalje svim ostalim procesima zamjenjuje tzv. *best-effort* semantika, gdje samo proizvoljni podskup svih ostalih procesa sazna za poruku koju je (i ako je) nekorektan pošiljatelj poslao difuzijom. Ovime lako dolazi do odstupanja između korektnih procesa - jedni znaju za poruku nekorektnog procesa, a drugi ne. Cilj seminara je opis, implementacija i demonstracija URB algoritma (*Uniform Reliable Broadcast*/uniformna pouzdana difuzija), koji osigurava da svi korektni procesi imaju isti podskup primljenih poruka, i njegovih CO i FIFO proširenja koja osiguravaju dodatna svojstva, ovisno o redoslijedu isporuke/kauzalnog poretka tih poruka, te dokazi osiguravanja spomenutih svojstava. Dodatno, pretpostavljamo pouzdanost kanala za razmjene poruka.

2 Distribuirani sustavi i komunikacijske apstrakcije

2.1 Distribuirani sustavi

Distribuirani sustav sastoji se od više procesora ili računala koji međusobno komuniciraju isključivo razmjenom poruka preko komunikacijske mreže. Za razliku od paralelnih sustava, distribuirani sustavi nemaju zajedničku memoriju, što znači da niti jedan proces ne može izravno pristupiti stanju drugog procesa niti imati trenutačni uvid u globalno stanje cijelog sustava.

Glavna prednost distribuiranih sustava je mogućnost raspodjele posla na više računala, čime se postiže veća skalabilnost i otpornost sustava na kvarove. Takvi sustavi danas se koriste u velikom broju aplikacija poput cloud servisa, bankarskih sustava, društvenih mreža i sustava za obradu velikih količina podataka.

Iako distribuirani sustavi omogućuju bolje iskorištavanje resursa i veću dostupnost, oni uvode i dodatne probleme koji ne postoje u centraliziranim sustavima. Procesi se mogu izvršavati različitim brzinama, mrežna komunikacija može kasniti, a pojedini procesi mogu prestati raditi zbog različitih kvarova. Takvo okruženje stvara veliku razinu nesigurnosti (*uncertainty*) jer procesi često ne mogu razlikovati radi li drugi proces sporo ili je došlo do njegovog kvara. Upravo zbog asinkronosti sustava, mogućih kvarova procesa i nepouzdanosti komunikacijskih kanala potrebno je koristiti posebne komunikacijske apstrakcije koje omogućuju pouzdanu razmjenu poruka među procesima.

2.2 Asinkrono okruženje

U asinkronim distribuiranim sustavima ne postoje garancije o vremenu izvršavanja procesa niti o vremenu potrebnom za isporuku poruka kroz mrežu. Poruka može biti isporučena gotovo trenutno, ali također može kasniti proizvoljno dugo. Isto tako, pojedini procesi mogu raditi sporije od ostalih zbog opterećenja sustava ili drugih vanjskih čimbenika.

Zbog takvog ponašanja proces ne može sa sigurnošću zaključiti je li drugi proces spor ili je došlo do njegovog kvara. To predstavlja jedan od glavnih izazova kod dizajna algoritama za distribuirane sustave. Algoritmi moraju biti sposobni raditi ispravno čak i kada postoje velika kašnjenja u komunikaciji ili kada dio procesa prestane s radom.

Asinkrono okruženje značajno otežava ostvarivanje pouzdane komunikacije jer ne postoji globalni sat niti mogućnost precizne sinkronizacije između procesa. To znači da poruke putuju konačno, ali proizvoljno dugo vremena, što one-

mogućuje razlikovanje vrlo sporog procesa od onog koji se srušio. Ovakvi sustavi nazivaju se "time-free" sustavima jer se napredak izračuna temelji isključivo na lokalnim događajima i redoslijedu primanja poruka, a ne na protoku fizičkog vremena. Događaji u asinkronom okruženju mogu se samo parcijalno urediti pomoću relacije "dogodilo-se-prije", koja se temelji na kauzalnosti slanja i primanja poruka. Zbog toga distribuirani algoritmi često koriste dodatne mehanizme poput potvrda primitka poruka, ponovnog slanja poruka i održavanja lokalnog stanja procesa.

2.3 Crash failure model

Jedan od najčešćih modela kvarova u distribuiranim sustavima je *crash failure* model koji pretpostavlja da proces čini kvar kada neočekivano i definitivno prestane s radom. Ključne karakteristike ovog modela su:

- Ispravan rad do kvara: Proces se ponaša točno prema algoritmu sve do trenutka rušenja.
- Bez pogrešnih radnji: Srušeni proces ne šalje "korumpirane" poruke niti poduzima korake koji odstupaju od njegovog koda (što je razlika u odnosu na bizantinske kvarove).
- Gornja granica kvarova (t): Parametar sustava koji označava maksimalni broj procesa koji se smiju srušiti u jednoj egzekuciji kako bi algoritmi ostali korektni.

Važno je naglasiti da se kod crash failure modela pretpostavlja kako proces prije kvara radi potpuno ispravno. Drugim riječima, proces ne šalje netočne ili zlonamjerne poruke, već jednostavno prestane s radom. Takav model kvarova znatno je jednostavniji od proizvoljnih ili bizantskih kvarova, ali i dalje predstavlja ozbiljan problem za pouzdanost distribuiranih sustava.

Kada proces prestane raditi tijekom komunikacije, može se dogoditi da samo dio procesa primi određenu poruku. To može dovesti do nekonzistentnog stanja sustava, odnosno situacije u kojoj različiti procesi posjeduju različite informacije. Zbog toga je potrebno koristiti algoritme koji garantiraju konzistentnu isporuku broadcast poruka svim ispravnim procesima.

2.4 Problem broadcast komunikacije

Broadcast komunikacija predstavlja mehanizam slanja jedne poruke svim procesima u sustavu. Obična operacija emitiranja, često označena kao **broadcast(m)**, definira se kao slanje kopije poruke svim procesima u sustavu. Kod osnovnog broadcast modela pošiljalac šalje poruku svim ostalim procesima, ali ne postoji garancija da će svi procesi poruku uspješno primiti.

Takva komunikacija ima semantiku *best effort* pristupa. Ako je pošiljalatelj ispravan, svi ispravni procesi će eventualno primiti poslanu poruku. Međutim, u prisutnosti *crash failure* kvarova može se dogoditi da se pošiljalatelj sruši tijekom slanja poruke. U tom slučaju samo proizvoljni podskup procesa može primiti poruku, dok ostali procesi za nju nikada ne saznaju.

Posljedica takvog ponašanja je nekonzistentnost sustava, odnosno situacija u kojoj različiti procesi posjeduju različite informacije o stanju sustava. Takvo ponašanje može uzrokovati ozbiljne probleme kod distribuiranih aplikacija koje zahtijevaju konzistentno stanje svih sudionika sustava i snažna jamstva o koordinaciji među procesima.

Kako bi se riješio taj problem, uvode se različite komunikacijske apstrakcije koje garantiraju određenu razinu pouzdanosti pri isporuci poruka. Cilj takvih apstrakcija je osigurati da svi ispravni procesi imaju konzistentan pogled na skup poslanih broadcast poruka čak i u prisutnosti kvarova procesa.

2.5 Komunikacijske apstrakcije broadcasta

Najjednostavniji oblik broadcast komunikacije naziva se *Best Effort Broadcast*. Kod ovog pristupa pošiljalatelj pokušava poslati poruku svim procesima, ali ne postoji garancija da će poruka zaista biti isporučena. Poruke se mogu izgubiti zbog problema u mreži ili zbog kvara procesa.

Kako bi se povećala pouzdanost komunikacije, koristi se *Reliable Broadcast*. Ova apstrakcija osigurava da će svi ispravni procesi eventualno primiti poruku koju je poslao ispravan proces. Time se postiže veća pouzdanost sustava, ali još uvijek mogu postojati problemi u situacijama kada neispravan proces isporuči poruku prije vlastitog kvara.

Uniform Reliable Broadcast (URB) dodatno proširuje svojstva pouzdanog broadcasta. Kod URB-a vrijedi pravilo da, ako bilo koji proces isporuči određenu poruku, tada će svi ispravni procesi također eventualno primiti tu poruku. Na taj način osigurava se da svi ispravni procesi završavaju s istim skupom broadcast poruka, čak i u prisutnosti crash failure kvarova.

Tip emitiranja	Jamstva isporuke
Best Effort Broadcast (BEB)	Jamči da će se poruka pokušati poslati svim procesima, ali ne postoji garancija da će je svi procesi primiti. U slučaju kvara pošiljatelja ili problema u mreži, poruka se može izgubiti.
Reliable Broadcast (RB)	Ako ispravan proces pošalje poruku, tada će je svi ispravni procesi eventualno primiti. Međutim, ako se proces sruši tijekom slanja, moguće je da samo dio procesa primi poruku.
Uniform Reliable Broadcast (URB)	Ako bilo koji proces primi poruku, tada svi ispravni procesi moraju eventualno primiti tu istu poruku. Time se osigurava da svi ispravni procesi imaju konzistentan skup primljenih poruka.

Tablica 1: Usporedba komunikacijskih apstrakcija broadcasta

2.6 Model sustava CAMP

CAMP (Crash Asynchronous Message-Passing) je standardni model distribuiranih sustava u kojem procesi komuniciraju isključivo slanjem poruka, bez zajedničke memorije, uz mogućnost crash failure kvarova.

Model se označava kao $\text{CAMP}_{n,t}[\emptyset]$, gdje n predstavlja ukupan broj procesa, a t maksimalan broj procesa koji se mogu srušiti tijekom izvođenja.

U asinkronom okruženju ne postoje ograničenja na kašnjenje poruka, pa se spor proces ne može razlikovati od srušenog procesa. Upravo zbog tih ograničenja, CAMP model se koristi kao standardni okvir za analizu komunikacijskih apstrakcija poput Best Effort Broadcasta, Reliable Broadcasta i Uniform Reliable Broadcasta.

Osnovni CAMP model može se proširiti dodatnim pretpostavkama (npr. $t < n/2$ ili detektori kvarova), čime se povećava njegova izražajna moć.

3 Uniformna pouzdana difuzija (URB)

Uniform Reliable Broadcast (URB) predstavlja ključnu komunikacijsku apstrakciju u distribuiranim sustavima koja omogućuje pouzdanu i konzistentnu razmjenu poruka u prisutnosti crash failure kvarova procesa. Osnovna ideja URB-a je osigurati da svi ispravni procesi na kraju sustava imaju isti skup primljenih poruka, neovisno o vremenskim kašnjenjima ili kvarovima pojedinih procesa.

URB apstrakcija definira dvije osnovne operacije:

- **URB_broadcast(m)** — operacija kojom proces šalje poruku m svim procesima u sustavu, uključujući i samog sebe.
- **URB_deliver(m)** — operacija kojom proces isporučuje (prima i prihvća) poruku m iz URB sustava.

3.1 Svojstva URB-a

URB se definira kroz četiri temeljna svojstva koja svaki algoritam implementacije mora zadovoljiti:

- **URB-validnost (Validity):** Ako ispravan proces isporuči poruku m , tada je ta poruka prethodno bila emitirana putem **URB_broadcast(m)** od strane nekog procesa. Ovim se sprječava stvaranje ili izmjena poruka koje nisu poslane.
- **URB-integritet (Integrity):** Svaki proces može isporučiti istu poruku najviše jednom. Time se osigurava da ne dolazi do dupliciranja poruka u sustavu.
- **URB-terminacija 1 (Termination-1):** Ako ispravan proces emitira poruku m , tada će taj proces eventualno i isporučiti poruku m .
- **URB-terminacija 2 (Uniform Agreement):** Ako bilo koji proces (bez obzira na to je li ispravan ili će se kasnije srušiti) isporučiti poruku m , tada svi ispravni procesi moraju eventualno isporučiti tu poruku.

3.2 Uniformnost

Svojstvo URB-validnosti povezuje isporučenu poruku s emitiranom porukom, odnosno izlaz s ulazom, čime se jamči da nema niti stvaranja novih, niti izmjene postojećih poruka. Dodatno, svojstvo URB-integriteta nalaže da ne dolazi do

dupliciranja poruka. Ova dva svojstva zajedno definiraju svojstvo sigurnosti (safety) URB-emitiranja. Važno je uočiti da su ona zadovoljena čak i u scenariju gdje se nijedna poruka nikada ne isporuči, bez obzira na to koje su poruke poslane.

Stoga je, kako bi specifikacija bila potpuna, nužno uvesti svojstvo živosti (liveness), odnosno osigurati da se poruke ne izgube nepovratno. To je svrha svojstva URB-terminacije: ako je proces koji emitira poruku ispravan (non-faulty) ili ako barem jedan proces (bio on ispravan ili neispravan) isporuči poruku m , tada tu poruku moraju isporučiti barem svi ispravni procesi. Zbog toga svojstva terminacije spadaju u porodicu "sve ili ništa" jamstava.

Drugim riječima, URB uvodi *uniformnost*: ako je bilo koji proces isporučio poruku, tada svi ispravni procesi moraju tu istu poruku također isporučiti. Ovo svojstvo eliminira mogućnost nekonzistentnog stanja među procesima i čini URB pogodnim za izgradnju koordinacijskih protokola u distribuiranim sustavima.

3.3 Važnost uniformnosti

Ključna razlika između Uniform Reliable Broadcast (URB) i običnog Reliable Broadcast (RB) nalazi se u svojstvu URB-terminacija 2. Kod RB-a je moguće da proces koji kasnije doživi crash failure isporuči poruku koju ostali ispravni procesi nikada ne prime. URB to strogo zabranjuje.

U nastavku se ne razmatra apstrakcija običnog RB-a jer nije korisna za praktične primjene. Budući da unaprijed nije poznato hoće li proces doživjeti crash failure ili ne, razumno je zahtijevati da se proces ponaša kao neispravan sve dok eventualno ne dođe do njegova rušenja.

3.4 Primjer

Lako je uočiti iz prethodne specifikacije da tijekom svakog izvođenja:

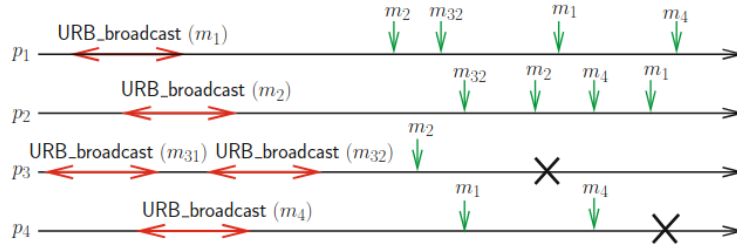
1. neispravni procesi isporučuju isti skup poruka,
2. taj skup uključuje sve poruke koje su poslali neispravni procesi, i
3. svaki neispravni proces isporučuje podskup poruka koje su isporučili ispravni procesi.

Dakle, dva različita neispravna procesa mogu isporučiti različite podskupove poruka. Također, važno je napomenuti da poruka m , koju je putem `urb-broadcast`

poslao neispravan proces, može biti ili ne biti **urb-delivered**. Nije moguće postaviti strogi zahtjev za njezinu isporuku, jer to ovisi o samom izvođenju sustava.

Jednostavan primjer prikazan je na Slici 1. Postoje četiri procesa koji putem **urb-broadcast** šalju pet poruka. Procesi p_1 i p_2 su ispravni, dok se procesi p_3 i p_4 ruše (što je prikazano prekrštenom strelicom). Isporuke poruka prikazane su okomitim zelenim strelicama duž osi procesa.

Oba procesa, p_1 i p_2 , putem **urb-deliver** isporučuju isti skup poruka $M = \{m_2, m_{32}, m_1, m_4\}$, dok svaki neispravan proces isporučuje podskup od M . Nadalje, poruka m_{31} , koju je putem **urb-broadcast** poslao neispravan proces p_3 , nikada nije **urb-delivered**, također p_3 ne isporučuje nijednu od poruka (m_{31} i m_{32}) koje je sam poslao putem **urb-broadcast**. Dodatno, poruka m_{32} , koju je p_3 poslao nakon m_{31} , isporučena je ispravnim procesima, dok m_{31} nije. To je posljedica zajedničkog učinka asinkronosti i rušenja procesa.



Slika 1: Primjer URB isporuke emitiranja

Dakle, lako je uočiti da isporuke poruka prikazane na Slici 1 zadovoljavaju specifikaciju uniformnog pouzdanog odašiljanja (*uniform reliable broadcast*).

3.5 URB paradigma

Problem uniformnog pouzdanog emitiranja (*uniform reliable broadcast*) predstavlja paradigmu koja obuhvaća obitelj problema koordinacije u distribuiranim sustavima. Kao primjer, naredbama **URB_broadcast** (m) i **URB_deliver** () mogu se pridružiti značenja „ovo je naredba” i „izvršavam je”. Iz toga slijedi da će svi ispravni procesi izvršiti isti skup naredbi (akcija). Taj skup uključuje sve naredbe koje su izdali ispravni procesi te može sadržavati i podskup naredbi koje su emitirali neispravni procesi.

Primijetimo da je URB-emitiranje problem jednokratnog izvođenja (*one-shot problem*). Specifikacija se primjenjuje na svaku poruku koja se emitira putem **URB_broadcast** neovisno o ostalim porukama koje su također emitirane putem **URB_broadcast**.

4 URB algoritam

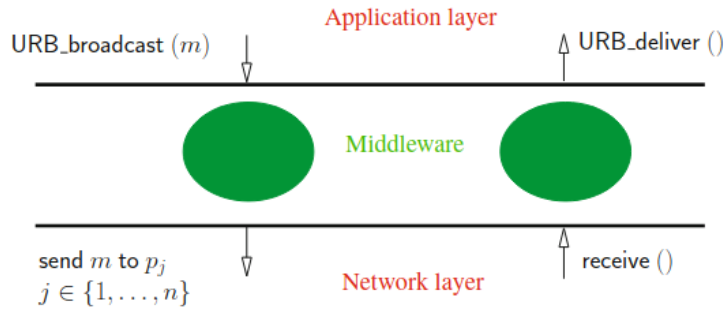
Algoritam koji implementira `URB_broadcast(m)` i `URB_deliver()` prikazan je na slici 2.

Operacija `URB_broadcast(m)` na strani klijenta započinje slanjem protokolarne poruke `MSG(m)` samome sebi od strane procesa p_i . Odnosno, kada proces p_i pozove operaciju `URB_broadcast(m)`, on najprije šalje poruku m samome sebi (linija 1), čime se osigurava uniformna obrada svih poruka, neovisno o njihovom izvoru.

Na poslužiteljskoj strani, kada proces p_i primi poruku `MSG(m)`, on provjerava je li poruku već ranije primio. Ako jest, poruka se odbacuje (linija 2). Ako je riječ o prvom primitku, proces je prosljeđuje (*rebroadcast*) svim ostalim procesima, osim sebi i izvornom pošiljatelju (linija 3), te tek zatim isporučuje poruku aplikacijskom sloju putem operacije `URB_deliver(m)` (linija 4).

(*Rebroadcast*) predstavlja temelj pouzdanosti jer jamči da će, čak i ako se izvorni pošiljatelj sruši, poruka nastaviti cirkulirati među ispravnim procesima sve dok je svi ne prime.

Ovakav pristup osigurava da se svaka poruka propagira kroz sustav putem *first-forward-then-deliver* strategije.



Slika 2: URB algoritam

Važno je primijetiti da proces p_i može ispreplitati izvršavanje više primitaka poruka istovremeno. Također, svakoj poruci dodaje se oznaka `MSG`. Poruka m naziva se *aplikacijska poruka* (*application message*), dok se poruka koja sadrži oznaku definiranu algoritmom, primjerice `MSG(m)`, naziva *protokolarna poruka* (*protocol message*).

4.1 Struktura podataka

Kako bi se spriječilo višestruko procesiranje iste poruke, svaka poruka m nosi jedinstveni identifikator oblika

$$\langle m.sender, m.seq_nb \rangle,$$

gdje $m.sender$ označava proces pošiljatelja, a $m.seq_nb$ lokalni sekvencijski broj poruke (1, 2, 3, ...).

Proces p_i održava lokalnu strukturu $rec_by_i[m]$, koja sadrži skup procesa za koje se zna da su primili poruku m .

4.2 Pseudokod algoritma

```
operation URB_broadcast(m) is
  (1) send MSG(m) to itself (process p_i).

when MSG(m) is received from process p_k do
  (2) if (first reception of message m) then
  (3)   for each j \in {1, . . . , n} \ {i, k} do
        send MSG(m) to process p_j
      end for;
  (4)   URB_deliver(m)
  (5) end if.
```

Napomena: u sustavima s nepouzdanim kanalima, linija (4) zamjenjuje se uvjetom:

$$|rec_by_i[m]| \geq t + 1.$$

4.3 Korektnost algoritma

Teorem 3. Algoritam implementira komunikacijsku apstrakciju URB-broadcast u sustavu $CAMP_{n,t}[\emptyset]$.

Dokaz. Svojstvo validnosti slijedi iz činjenice da se isporučuju samo poruke koje su prethodno zaprimljene.

Svojstvo integriteta slijedi iz toga da se svaka poruka isporučuje najviše jednom, i to pri prvom primitku.

Svojstvo terminacije proizlazi iz strategije prosljeđivanja: svaki proces koji primi poruku, prosljeđuje je dalje svim ostalim procesima.

Ako ispravan proces emitira poruku m , on će je poslati svim procesima, te će zbog pouzdanih kanala svaki proces na kraju primiti i isporučiti poruku. Ako bilo koji proces isporuči poruku m , tada je on prethodno proslijedio tu poruku, pa se isti argument propagacije primjenjuje na cijeli sustav.

4.4 ACK potvrde i uvjet za isporuku (deliver)

U osnovnom modelu s pouzdanim kanalima ($\text{CAMP}_{n,t}[\emptyset]$), proces isporučuje poruku aplikacijskom sloju odmah nakon što ju je proslijedio drugim procesima. Međutim, u sustavima s nepouzdanim kanalima, proces koristi ACK potvrde kako bi pratio koji su procesi primili poruku.

U tom modelu uvjet za isporuku jest da proces zna kako je barem $t + 1$ procesa primilo kopiju poruke. Budući da se najviše t procesa može srušiti, ovaj uvjet jamči da je poruku primio barem jedan ispravan proces koji će je nastaviti retransmitirati sve dok ne stigne do svih procesa.

5 Proširenja algoritma

Uniformni pouzdani broadcast (Uniform Reliable Broadcast – URB) daje jamstva o tome koje će poruke biti isporučene procesima. Svi ispravni procesi isporučuju isti skup poruka M , dok svaki neispravni proces p_i može isporučiti samo podskup $M_i \subseteq M$.

Neke je distribuirane aplikacije lakše projektirati kada sustav pruža dodatna jamstva o redoslijedu isporuke poruka. Takva jamstva odnose se na redoslijed kojim se poruke predaju aplikaciji višeg sloja. U ovom radu razmatramo dva svojstva:

- **FIFO (First In, First Out)** svojstvo,
- **CO (Causal Order)** odnosno uzročno uređeno svojstvo.

Modularni prikaz odnosa između ovih apstrakcija prikazan je na slici 3. Svaka strelica predstavlja konstrukciju kojom se jedna komunikacijska apstrakcija gradi na temelju druge. Važno je naglasiti da se FIFO-URB i CO-URB mogu implementirati u svakom sustavu u kojem je moguće implementirati URB. U usporedbi s URB-om, ove apstrakcije ne zahtijevaju dodatne pretpostavke o modelu sustava niti dodatne mehanizme poput detektora kvarova.



Slika 3: Enter Caption

URB-broadcast predstavlja *one-shot* problem jer se svaka poruka promatra neovisno o svim ostalim porukama. Svojstva i jamstva definiraju se za pojedinačnu poruku bez potrebe za razmatranjem njezina odnosa s drugim porukama.

S druge strane, FIFO-URB i CO-URB predstavljaju *multi-shot* probleme. Njihove specifikacije uključuju međusobne odnose između više poruka koje se šalju komunikacijskim kanalima. Kod FIFO svojstva potrebno je očuvati redoslijed slanja poruka istog pošiljatelja, dok kod uzročno uređene isporuke sustav mora očuvati uzročno-posljedične veze među porukama.

5.1 FIFO

Apstrakcija FIFO-URB temelji se na dvjema osnovnim operacijama: `FIFO broadcast(m)` i `FIFO deliver()`. Ona predstavlja proširenje apstrakcije Uniform Reliable Broadcast (URB), koja osigurava svojstva valjanosti, integriteta i terminacije opisana u prethodnom poglavlju. Uz navedena svojstva, FIFO-URB uvodi dodatno jamstvo koje se odnosi na redoslijed isporuke poruka, odnosno URB-FIFO ima sljedeće dodatno svojstvo:

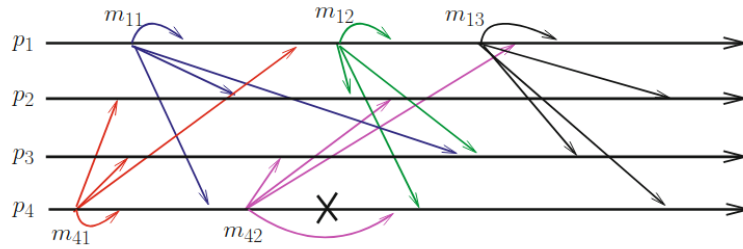
- **FIFO-URB isporuka poruka.** Ako proces najprije FIFO-broadcasta poruku m , a zatim FIFO-broadcasta poruku m' , nijedan proces ne smije FIFO-isporučiti poruku m' prije nego što je prethodno FIFO-isporučio poruku m .

Ovo svojstvo osigurava da se poruke koje šalje svaki proces promatraju zasebno te da se isporučuju u istom redoslijedu u kojem su poslone. Ne postoje ograničenja vezana uz međusobni redoslijed poruka koje šalju različiti procesi.

Važno je primijetiti da FIFO-URB svojstvo isporuke sprječava neispravan proces da FIFO-isporučiti poruku m' bez prethodne isporuke poruke m . Za bilo koji proces p_i , neispravan proces može FIFO-isporučiti samo prefiks poruka koje je poslao p_i .

5.1.1 Primjer

Jednostavan primjer prikazan je na slici 4. Proces p_1 FIFO-broadcasta poruke m_{11} , zatim m_{12} i na kraju m_{13} . Proces p_4 FIFO-broadcasta poruke m_{41} i zatim m_{42} .



Slika 4: Enter Caption

FIFO-URB svojstvo zahtijeva da svaka instanca sustava isporuči poruku m_{11}

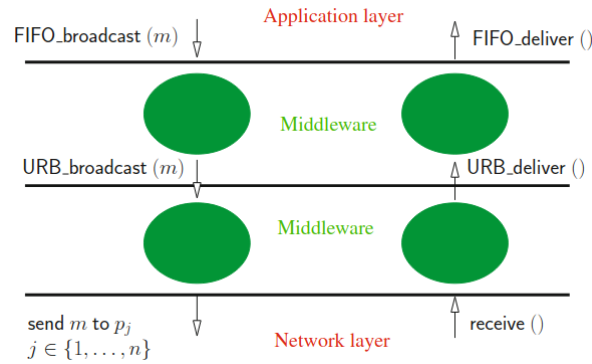
prije poruke m_{12} , te poruku m_{12} prije poruke m_{13} . Analogno tome, nijedan proces ne smije isporučiti poruku m_{42} prije poruke m_{41} .

U prikazanom primjeru proces p_4 prestaje s radom prije nego što uspije FIFO-isporučiti vlastitu poruku m_{42} .

Budući da FIFO-URB ne nameće ograničenja nad porukama koje šalju različiti procesi, poruke procesa p_1 i p_4 mogu biti međusobno isprepletene na različite načine kod različitih primatelja.

5.1.2 Jednostavna konstrukcija

Konstrukcija pretpostavlja da komunikacijski sloj pruža apstrakciju Uniform Reliable Broadcast (URB).



Slika 5: Enter Caption

Jednostavan način implementacije FIFO svojstva sastoji se u pridruživanju odgovarajućeg predikata svakoj poruci. Dok je predikat netočan, poruka ostaje pohranjena u ulaznom međuspremniku procesa. Čim predikat postane istinit, poruka se isporučuje aplikaciji.

Svaki proces p_i koristi dvije lokalne varijable:

- msg_set_i . – skup poruka koje su URB-isporučene, ali još nisu FIFO-isporučene.
- $\text{next}_i[1..n]$ – polje koje za svaki proces p_j sadrži redni broj sljedeće poruke koju proces p_i očekuje FIFO-isporučiti od procesa p_j .

Prva poruka koju proces pošalje dobiva redni broj 1, druga redni broj 2 itd.

Operacija **FIFO broadcast**(m) jednostavno poziva operaciju **URB broadcast**(m).

Kada proces URB-isporuči poruku m , provjerava njezin redni broj:

- Ako je stigla očekivana poruka, ona se odmah FIFO-isporučuje.
- Ako je poruka stigla prerano (odnosno prethodne poruke još nisu isporučene), sprema se u skup msg_set_i .

Nakon FIFO-ispоруke neke poruke, proces provjerava postoje li u međuspremniku sljedeće poruke istog pošiljatelja koje sada mogu biti isporučene te ih redom isporučuje.

Algoritam.

operation *FIFO_broadcast*(m) **is**

- (1) $m.sender \leftarrow i$; $m.seq_nb \leftarrow p_i$'s next sequence number (starting from 1);
- (2) *URB_broadcast* *MSG*(m).

when *MSG*(m) **is** urb-delivered **do**

- (3) let $j = m.sender$;
- (4) **if** ($next_i[j] = m.seq_nb$)
- (5) **then** *FIFO_deliver*(m);
- (6) $next_i[j] \leftarrow next_i[j] + 1$;
- (7) **while** ($\exists m' \in msg_set_i : (m'.sender = j) \wedge (next_i[j] = m'.seq_nb)$)
- (8) **do** *FIFO_deliver*(m');
- (9) $next_i[j] \leftarrow next_i[j] + 1$;
- (10) $msg_set_i \leftarrow msg_set_i \setminus \{m'\}$;
- (11) **end while**
- (12) **else** $msg_set_i \leftarrow msg_set_i \cup \{m\}$;
- (13) **end if**.

5.1.3 Teorem

Teorem 4. Algoritam implementira FIFO-URB komunikacijsku apstrakciju u svakom sustavu u kojem je moguće implementirati URB-broadcast.

Dokaz Dokaz izravno slijedi iz svojstava osnovne URB-broadcast apstrakcije (Teorem 3) te korištenja rednih brojeva poruka. Budući da URB jamči pouzdanu i jedinstvenu isporuku poruka svim ispravnim procesima, a redni brojevi omogućuju provjeru njihovog redoslijeda, dobiva se FIFO svojstvo isporuke prema kojem se poruke svakog pošiljatelja isporučuju točno redoslijedom kojim su poslone.

5.2 CO-URB

Promatramo komunikacijsku apstrakciju koja se naziva *Uniformna pouzdana difuzija s kauzalnim redoslijedom*, odnosno *CO-URB*.

Osnovni URB osigurava da svi ispravni procesi na kraju isporuče isti skup poruka. Međutim, osnovni URB ne propisuje redoslijed u kojem se te poruke moraju isporučiti. Kod CO-URB-a uvodi se dodatni zahtjev: ako je neka poruka kauzalno prethodila drugoj poruci, tada svaka isporuka mora poštovati taj redoslijed.

5.2.1 Parcijalni poredak nad porukama

Neka je M skup svih poruka koje su CO-broadcastane tijekom neke izvedbe sustava. Na tom skupu definiramo relaciju kauzalnog prethodenja, koju označavamo $s \rightarrow_M$.

Za dvije poruke $m, m' \in M$ kažemo da poruka m kauzalno prethodi poruci m' , i pišemo

$$m \rightarrow_M m',$$

ako vrijedi barem jedan od sljedećih uvjeta:

1. Poruke m i m' CO-broadcastao je isti proces, pri čemu je m broadcastana prije poruke m' .
2. Neki proces p_i najprije je CO-isporučio poruku m , a zatim je CO-broadcastao poruku m' .
3. Postoji poruka $m'' \in M$ takva da vrijedi

$$m \rightarrow_M m'' \quad \text{i} \quad m'' \rightarrow_M m'.$$

Treći uvjet predstavlja tranzitivnost kauzalnog prethodenja. Dakle, ako je poruka m kauzalno prije poruke m'' , a poruka m'' kauzalno prije poruke m' , tada je i poruka m kauzalno prije poruke m' .

Relacija $\rightarrow_M$ definira parcijalni poredak nad porukama. To znači da neke poruke moraju biti isporučene u određenom redoslijedu, dok neke poruke mogu biti međusobno neovisne. Ako za dvije poruke m i m' ne vrijedi ni

$$m \rightarrow_M m' \quad \text{ni} \quad m' \rightarrow_M m,$$

tada su te poruke kauzalno neovisne i procesi ih smiju isporučiti u različitim redoslijedima.

5.2.2 Kauzalna isporuka poruka

CO-URB komunikacijska apstrakcija sastoji se od dvije operacije:

$$\text{CO-broadcast}(m) \quad \text{i} \quad \text{CO-deliver}(m).$$

Osim svojstava koja vrijede za URB, kao što su valjanost, integritet i terminacija, CO-URB mora zadovoljiti i dodatno svojstvo kauzalnog redoslijeda.

Svojstvo CO-URB isporuke poruka. Ako za dvije poruke m i m' vrijedi $m \rightarrow_M m'$, tada nijedan proces ne smije CO-isporučiti poruku m' prije nego što je prethodno CO-isporučio poruku m .

Drugim riječima, ako poruka m' kauzalno ovisi o poruci m , tada svaki proces koji isporuči obje poruke mora poruku m isporučiti prije poruke m' .

5.2.3 Odnos FIFO i CO redoslijeda

FIFO redoslijed može se promatrati kao poseban slučaj kauzalnog redoslijeda. FIFO zahtijeva da se poruke istog pošiljatelja isporučuju redoslijedom kojim su poslone.

Ako proces p_i najprije broadcasta poruku m , a zatim poruku m' , tada FIFO zahtijeva da svaki proces isporuči m prije m' .

CO redoslijed proširuje ovu ideju. On ne promatra samo poruke istog pošiljatelja, nego i poruke različitih pošiljatelja ako između njih postoji kauzalna veza.

Primjerice, ako proces p_1 pošalje poruku m_1 , zatim proces p_2 isporuči m_1 , a nakon toga proces p_2 pošalje poruku m_2 , tada vrijedi

$$m_1 \rightarrow_M m_2.$$

Ovdje su poruke poslali različiti procesi, ali svejedno postoji kauzalna ovisnost. Zato CO redoslijed zahtijeva da m_1 bude isporučena prije m_2 .

Dakle, FIFO redoslijed odnosi se samo na redoslijed poruka istog pošiljatelja, dok CO redoslijed obuhvaća i kauzalne veze između poruka različitih pošiljatelja.

Za opis kauzalnog redoslijeda korisno je uvesti i svojstvo lokalnog redoslijeda.

Lokalni redoslijed. Ako neki proces najprije primi poruku m , a zatim broadcasta poruku m' , tada nijedan proces ne smije primiti poruku m' prije nego što je primio poruku m .

Formalno, ako proces p_i izvede

$$\text{CO-deliver}(m) \text{ prije } \text{CO-broadcast}(m'),$$

tada svaki proces koji primi m' mora prije toga primiti m .

Ovo svojstvo opisuje ideju da nova poruka koju proces šalje može ovisiti o svim porukama koje je taj proces prethodno dostavio.

Kauzalni redoslijed može se karakterizirati pomoću dva jednostavnija svojstva:

$$\text{CO redoslijed} \equiv \text{FIFO redoslijed} + \text{lokalni redoslijed}.$$

To znači da je za dokaz da algoritam zadovoljava kauzalni redoslijed dovoljno pokazati sljedeće:

1. algoritam zadovoljava FIFO redoslijed, tj. poruke istog pošiljatelja isporučuju se redoslijedom kojim su poslone;
2. algoritam zadovoljava lokalni redoslijed, tj. ako proces dostavi neku poruku prije slanja nove poruke, tada ta dostavljena poruka mora biti isporučena prije nove poruke kod svakog procesa.

Ako su oba svojstva zadovoljena, tada je zadovoljen i kauzalni redoslijed.

Teorem 1. *Kauzalni redoslijed isporuke poruka ekvivalentan je kombinaciji FIFO redoslijeda i lokalnog redoslijeda.*

Dokaz. Iz same definicije kauzalnog redoslijeda slijedi da svojstvo kauzalnog redoslijeda implicira FIFO redoslijed i lokalni redoslijed.

Naime, ako isti proces najprije broadcasta poruku m , a zatim poruku m' , tada prema definiciji kauzalnog prethodenja vrijedi

$$m \rightarrow_M m'.$$

Zbog svojstva kauzalnog redoslijeda, svaki proces koji isporuči m' mora prethodno isporučiti m . Time je zadovoljen FIFO redoslijed.

Također, ako neki proces najprije isporuči poruku m , a zatim broadcasta poruku m' , tada ponovno prema definiciji kauzalnog prethodenja vrijedi

$$m \rightarrow_M m'.$$

Zbog svojstva kauzalnog redoslijeda, svaki proces koji isporuči m' mora prethodno isporučiti m . Time je zadovoljen lokalni redoslijed.

Preostaje pokazati obratni smjer. Pretpostavimo da su zadovoljena svojstva FIFO redoslijeda i lokalnog redoslijeda. Neka su m i m' dvije poruke takve da vrijedi

$$m \rightarrow_M m',$$

i neka je p proces koji isporučuje poruku m' . Trebamo pokazati da proces p isporučuje poruku m prije poruke m' .

Budući da vrijedi $m \rightarrow_M m'$, postoji konačan niz poruka

$$m = m_1, m_2, \dots, m_{k-1}, m_k = m',$$

gdje je $k \geq 2$, takav da su te poruke broadcastali procesi

$$q_1, q_2, \dots, q_k,$$

te za svaki x , gdje je $1 \leq x < k$, vrijedi

$$m_x \rightarrow_M m_{x+1}.$$

Pritom svaki prijelaz $m_x \rightarrow_M m_{x+1}$ nastaje zbog jednog od prva dva pravila iz definicije kauzalnog prethodenja, odnosno ne promatramo posebno tranzitivno pravilo jer je ono već obuhvaćeno samim nizom poruka.

Za svaki x takav da je $1 \leq x < k$ razlikujemo dva slučaja.

- Ako je $q_x = q_{x+1}$, tada su poruke m_x i m_{x+1} broadcastane od istog procesa. Budući da je m_x broadcastana prije poruke m_{x+1} , iz FIFO redoslijeda slijedi da proces p mora isporučiti m_x prije m_{x+1} .
- Ako je $q_x \neq q_{x+1}$, tada su poruke m_x i m_{x+1} broadcastane od različitih procesa. U tom slučaju proces q_{x+1} isporučio je poruku m_x prije nego što je broadcastao poruku m_{x+1} . Iz svojstva lokalnog redoslijeda slijedi da proces p mora isporučiti m_x prije m_{x+1} .

Dakle, za svaki susjedni par poruka u nizu vrijedi da ih proces p isporučuje u redoslijedu

$$m_x \text{ prije } m_{x+1}.$$

Posebno, kada proces p isporučuje poruku $m_k = m'$, on je prethodno morao isporučiti poruku m_{k-1} . Isto tako, prije isporuke poruke m_{k-1} morao je isporučiti poruku m_{k-2} , i tako redom sve do poruke $m_1 = m$.

Prema tome, ako proces p isporuči poruku m' , tada je prethodno isporučio poruku m . Time je pokazano da FIFO redoslijed zajedno s lokalnim redoslijedom implicira kauzalni redoslijed.

Zaključujemo da je kauzalni redoslijed ekvivalentan kombinaciji FIFO redoslijeda i lokalnog redoslijeda. $\square$

5.2.4 Jednostavna konstrukcija CO-broadcasta iz URB-broadcasta

Prije konstrukcije CO-broadcasta koja se temelji na FIFO-broadcast apstrakciji, može se promatrati jednostavnija konstrukcija CO-broadcasta izravno nad URB-broadcastom. Ova konstrukcija je jednostavna za razumjeti, ali je vrlo neučinkovita.

Neka je m aplikacijska poruka koju proces želi CO-broadcastati. Osnovna ideja algoritma je da se ne šalje samo poruka m , nego protokolarna poruka koja sadrži poruku m zajedno sa svim porukama koje joj kauzalno prethode. Drugim riječima, svaka poslana poruka nosi svoju kauzalnu prošlost.

Svaki proces p_i održava lokalnu varijablu $causal_past_i$, koja sadrži niz svih poruka koje kauzalno prethode sljedećoj poruci koju će proces p_i CO-broadcastati. Formalno, ako je m sljedeća poruka koju će proces p_i poslati, tada $causal_past_i$ sadrži sve poruke m' za koje vrijedi $m' \rightarrow_M m$.

Na početku je $causal_past_i = \epsilon$, gdje ϵ označava prazan niz.

Operator $\oplus$ označava dodavanje poruke na kraj niza. Dakle,

$$causal_past_i \oplus m$$

označava niz koji se dobije tako da se na kraj niza $causal_past_i$ doda poruka m .

Algoritam.

```

operation CO_broadcast( $m$ ) is
  (1) URB_broadcast MSG( $causal\_past_i \oplus m$ );
  (2)  $causal\_past_i \leftarrow causal\_past_i \oplus m$ .

when MSG( $\langle m_1, \dots, m_\ell \rangle$ ) is urb-delivered do
  (3) for  $x$  from 1 to  $\ell$  do
  (4)   if  $m_x$  has not yet been CO-delivered then
  (5)     CO_deliver( $m_x$ );
  (6)      $causal\_past_i \leftarrow causal\_past_i \oplus m_x$ ;
  (7)   end if
  (8) end for.

```

Kada proces p_i želi CO-broadcastati poruku m , on pomoću URB-a šalje protokolarnu poruku *MSG*($causal_past_i \oplus m$). Ta protokolarna poruka sadrži sve poruke iz njegove kauzalne prošlosti, zajedno s novom porukom m . Nakon toga proces ažurira svoju lokalnu varijablu: $causal_past_i \leftarrow causal_past_i \oplus m$. Time poruka m postaje dio kauzalne prošlosti svih budućih poruka koje će proces p_i CO-broadcastati.

Kada proces p_i URB-isporuči protokolarnu poruku $MSG(\langle m_1, \dots, m_\ell \rangle)$, on redom prolazi kroz sve aplikacijske poruke u tom nizu. Za svaku poruku m_x provjerava je li već bila CO-isporučena. Ako jest, poruka se ignorira. Ako nije, proces je CO-isporuči i dodaje na kraj svoje lokalne kauzalne prošlosti: $causal_past_i \leftarrow causal_past_i \oplus m_x$.

Važno je da se poruke iz primljenog niza obrađuju upravo redom kojim se nalaze u nizu. Budući da niz sadrži kauzalnu prošlost poruke prije same poruke, time se osigurava da će svaka poruka biti isporučena tek nakon svih poruka koje joj kauzalno prethode.

Zašto algoritam osigurava kauzalni redoslijed? Pretpostavimo da vrijedi $m' \rightarrow_M m$. To znači da poruka m' kauzalno prethodi poruci m . Prema konstrukciji algoritma, kada se broadcasta poruka m , njezina kauzalna prošlost već se nalazi u nizu $causal_past_i$. Zato se u protokolarnoj poruci šalje niz $causal_past_i \oplus m$, u kojem se poruka m' pojavljuje prije poruke m .

Kada neki proces primi taj niz, on ga obrađuje redom. Stoga će najprije CO-isporučiti poruke iz kauzalne prošlosti, a tek nakon njih poruku m . Ako je neku od tih poruka već prije CO-isporučio, tada je preskače, ali time se kauzalni redoslijed ne narušava.

Prema tome, nijedan proces ne može CO-isporučiti poruku m prije poruke m' ako vrijedi $m' \rightarrow_M m$.

Neučinkovitost algoritma. Glavni nedostatak ove konstrukcije je neučinkovitost. Varijabla $causal_past_i$ stalno raste, jer se u nju dodaje svaka nova CO-isporučena poruka. Zbog toga i protokolarnе poruke koje se šalju postaju sve veće. Ako je proces tijekom vremena dostavio mnogo poruka, tada će svaka njegova sljedeća poruka sadržavati vrlo velik niz prethodnih poruka. Dakle, veličina protokolarnih poruka raste neograničeno.

Zato je ova konstrukcija korisna za razumijevanje ideje kauzalnog redoslijeda, ali nije praktična za učinkovitu implementaciju.

5.2.5 Konstrukcija CO-broadcasta iz FIFO-broadcasta

U prethodnoj konstrukciji CO-broadcasta iz URB-broadcasta svaka je protokolarna poruka nosila cijelu kauzalnu prošlost poruke koja se šalje. Takva konstrukcija je jednostavna, ali vrlo neučinkovita jer veličina protokolarnih poruka raste neograničeno.

U ovom dijelu promatra se učinkovitija konstrukcija CO-broadcasta koja se te-

melji na FIFO-broadcast apstrakciji. Osnovna ideja ove konstrukcije proizlazi iz činjenice da FIFO-broadcast već pamti redoslijed poruka koje je poslao isti proces. Drugim riječima, FIFO-broadcast osigurava da se poruke istog pošiljatelja isporučuju redoslijedom kojim su broadcastane.

Zbog toga proces više ne mora u svakoj poruci slati cijelu svoju kauzalnu prošlost. Umjesto toga, dovoljno je poslati samo one poruke koje je proces CO-isporučio od svojeg prethodnog CO-broadcasta.

Svaki proces p_i održava lokalnu varijablu $im_causal_past_i$. Ta varijabla sadrži niz poruka koje je proces p_i CO-isporučio od svojeg prethodnog CO-broadcasta. Na početku je $im_causal_past_i = \epsilon$, gdje ϵ označava prazan niz. Za razliku od prethodne konstrukcije, ovdje se ne čuva cijela kauzalna prošlost, nego samo njezin sufiks, odnosno samo najnoviji dio koji još nije bio poslan u nekoj prethodnoj CO-broadcast poruci.

Algoritam. Kada proces p_i želi CO-broadcastati poruku m , on pomoću FIFO-broadcasta šalje protokolarnu poruku koja sadrži niz $im_causal_past_i \oplus m$. Nakon toga se lokalna varijabla $im_causal_past_i$ resetira na prazan niz.

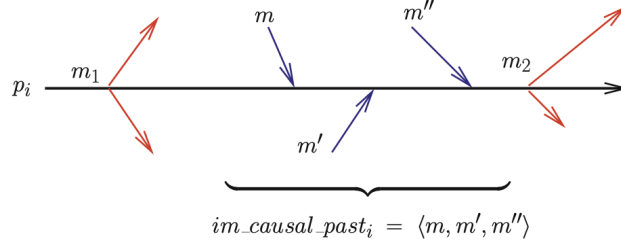
operation $CO_broadcast(m)$ **is**
 (1) $FIFO_broadcast\ MSG(im_causal_past_i \oplus m)$;
 (2) $im_causal_past_i \leftarrow \epsilon$.

Kada proces p_i FIFO-isporuči protokolarnu poruku oblika $MSG(\langle m_1, \dots, m_\ell \rangle)$, on redom prolazi kroz sve aplikacijske poruke u tom nizu. Ako neka poruka još nije bila CO-isporučena, proces je CO-isporuči i dodaje u svoj lokalni niz $im_causal_past_i$.

when $MSG(\langle m_1, \dots, m_\ell \rangle)$ **is FIFO-delivered do**
 (3) **for** x **from** 1 **to** ℓ **do**
 (4) **if** m_x has not yet been CO-delivered **then**
 (5) $CO_deliver(m_x)$;
 (6) $im_causal_past_i \leftarrow im_causal_past_i \oplus m_x$;
 (7) **end if**
 (8) **end for.**

Objašnjenje rada algoritma. Pretpostavimo da proces p_i najprije CO-broadcasta poruku m_1 , a zatim kasnije CO-broadcasta poruku m_2 .

Između ta dva broadcasta proces p_i može CO-isporučiti neke druge poruke. Neka su to poruke m, m', m'' , i neka ih je proces p_i CO-isporučio upravo tim redoslijedom.



Slika 6: Enter Caption

Tada neposredno prije slanja poruke m_2 vrijedi $im_causal_past_i = \langle m, m', m'' \rangle$. Kada proces p_i sada CO-broadcasta poruku m_2 , on zapravo FIFO-broadcasta protokolarnu poruku $MSG(\langle m, m', m'', m_2 \rangle)$. Time proces p_i svim ostalim procesima javlja da poruke m, m', m'' moraju biti CO-isporučine prije poruke m_2 , ako već nisu bile CO-isporučine.

Nakon slanja te protokolarne poruke, proces p_i resetira svoju lokalnu varijablu: $im_causal_past_i \leftarrow \epsilon$. To je dopušteno zato što je informacija o porukama m, m', m'' već uključena u upravo poslanu protokolarnu poruku.

Zašto je ovdje potreban FIFO-broadcast? Ključna razlika u odnosu na prethodnu konstrukciju jest u tome što se ovdje oslanjamo na FIFO-broadcast.

FIFO-broadcast osigurava da ako isti proces najprije pošalje jednu protokolarnu poruku, a zatim drugu, svi procesi ih dostavljaju tim istim redoslijedom.

Zato proces p_i ne mora stalno slati cijelu kauzalnu prošlost. Poruke koje su već bile uključene u neku raniju protokolarnu poruku ne moraju se ponovno slati, jer FIFO-broadcast osigurava da će ta ranija protokolarna poruka biti dostavljena prije kasnije protokolarne poruke istog pošiljatelja.

Drugim riječima, FIFO-broadcast već čuva dio informacije o redoslijedu, pa se u protokolarnoj poruci mora slati samo ono što je novo od zadnjeg CO-broadcasta.

Usporedba s prethodnom konstrukcijom. U jednostavnoj konstrukciji iz URB-broadcasta proces je u svakoj novoj poruci slao cijelu varijablu $causal_past_i$. Ta varijabla je stalno rasla, jer je sadržavala sve poruke koje je proces ikada CO-isporučio.

U FIFO-broadcast konstrukciji proces koristi varijablu $im_causal_past_i$, koja sadrži samo poruke dostavljene od zadnjeg CO-broadcasta. Nakon svakog CO-broadcasta ta se varijabla resetira na prazan niz.

Zato je ova konstrukcija učinkovitija od prethodne.

Zaključak. Konstrukcija CO-broadcasta iz FIFO-broadcasta temelji se na ideji da FIFO-broadcast već osigurava redoslijed poruka istog pošiljatelja. Zbog toga nije potrebno u svakoj poruci slati cijelu kauzalnu prošlost. Dovoljno je poslati samo poruke koje su dostavljene od prethodnog CO-broadcasta.

Time se lokalna varijabla $causal_past_i$ zamjenjuje varijablom $im_causal_past_i$, koja se nakon svakog CO-broadcasta resetira na prazan niz. Na taj se način smanjuje količina informacija koje se šalju, a pritom se i dalje osigurava kauzalni redoslijed isporuke poruka.

Teorem 2. *Opisan algoritam gradi komunikacijsku apstrakciju CO-URB-broadcast u svakom sustavu u kojem se može izgraditi FIFO-URB-broadcast.*

Dokaz. Dokazujemo da algoritam zadovoljava svojstva CO-URB-broadcast apstrakcije. To su svojstva valjanosti, integriteta, terminacije i kauzalnog redoslijeda isporuke poruka.

Valjanost i integritet. Najprije primijetimo da je operacija $CO_broadcast(m)$ implementirana pomoću FIFO-broadcasta. Zbog toga algoritam izravno nasljeđuje svojstvo valjanosti FIFO-broadcasta: protokolarnе poruke se ne stvaraju proizvoljno i ne mijenjaju se tijekom prijenosa.

Također, algoritam nasljeđuje i svojstvo integriteta FIFO-broadcasta: svaka protokolarna poruka može biti FIFO-isporučena najviše jednom.

Budući da protokolarna poruka sadrži aplikacijske poruke, iz toga slijedi da se nijedna aplikacijska poruka ne može izgubiti ili promijeniti. Osim toga, prije svake CO-isporuke aplikacijske poruke algoritam provjerava je li ta poruka već bila CO-isporučena. Zbog te provjere nijedna aplikacijska poruka ne može biti CO-isporučena više od jednom.

Time su zadovoljena svojstva valjanosti i integriteta.

Terminacija. Kada proces CO-broadcasta aplikacijsku poruku m , on zapravo FIFO-broadcasta protokolarnu poruku oblika $MSG(seq \oplus m)$, gdje je seq niz aplikacijskih poruka koje predstavljaju dio kauzalne prošlosti poruke m .

Kada proces FIFO-isporuči protokolarnu poruku $MSG(\langle m_1, \dots, m_\ell \rangle)$, tada svaku aplikacijsku poruku m_x , za $1 \leq x \leq \ell$, odmah CO-isporuči ako ona još nije bila CO-isporučena.

Dakle, algoritam ne odgađa CO-isporuku aplikacijskih poruka koje se nalaze u FIFO-isporučenoj protokolarnoj poruci. Zbog toga algoritam nasljeđuje svojstvo terminacije od FIFO-broadcasta. Budući da FIFO-broadcast osigurava da će svaka FIFO-broadcastana protokolarna poruka biti FIFO-isporučena svim ispravnim procesima, slijedi da će svaka CO-broadcastana aplikacijska poruka biti CO-isporučena.

Time je zadovoljena terminacija.

Kauzalni redoslijed isporuke. Preostaje dokazati svojstvo kauzalnog redoslijeda. Trebamo pokazati da za svake dvije poruke m i m' takve da vrijedi $m \rightarrow_M m'$, nijedan proces ne može CO-isporučiti poruku m' prije nego što je prethodno CO-isporučio poruku m .

Dokaz se temelji na tri tvrdnje.

Tvrdnja C1. Pretpostavimo da proces p_i FIFO-broadcasta protokolarnu poruku $MSG(seq' \oplus m')$, gdje je seq' niz aplikacijskih poruka. Pretpostavimo dodatno da vrijedi barem jedan od sljedeća dva uvjeta: $m \in seq'$ ili je proces p_i ranije FIFO-broadcastao protokolarnu poruku $MSG(seq \oplus m)$. Tada nijedan proces ne može CO-isporučiti poruku m' prije nego što je prethodno CO-isporučio poruku m .

Dokaz tvrdnje C1. Dokazujemo tvrdnju kontradikcijom. Pretpostavimo suprotno: neka postoji proces koji CO-isporuči m' prije nego što je CO-isporučio m . Neka je τ prvi trenutak u kojem neki proces CO-isporuči m' bez da je prethodno CO-isporučio m . Neka je p_j takav proces.

Promotrimo što je uzrokovalo da proces p_j CO-isporuči poruku m' .

Slučaj 1. Proces p_j FIFO-isporučio je protokolarnu poruku $MSG(seq' \oplus m')$. Razlikujemo dva podslučaja.

- Ako je $m \in seq'$, tada se u protokolarnoj poruci $MSG(seq' \oplus m')$ poruka m pojavljuje prije poruke m' . Budući da algoritam obrađuje poruke u nizu redom, proces p_j mora CO-isporučiti m prije m' .
- Ako je proces p_i ranije FIFO-broadcastao $MSG(seq \oplus m)$, a zatim FIFO-broadcastao $MSG(seq' \oplus m')$, tada iz FIFO svojstva slijedi da proces p_j mora FIFO-isporučiti $MSG(seq \oplus m)$ prije $MSG(seq' \oplus m')$. Prema tekstu algoritma, iz prve protokolarne poruke proces p_j CO-isporuči m prije nego što iz druge protokolarne poruke može CO-isporučiti m' .

U oba podslučaja dobivamo da p_j CO-isporuči m prije m' , što je kontradikcija s pretpostavkom da je p_j CO-isporučio m' prije m .

Slučaj 2. Proces p_j CO-isporučio je m' zato što je FIFO-isporučio neku drugu protokolarnu poruku $MSG(seq'' \oplus m'')$, takvu da je $m' \in seq''$, a poruka m se u tom nizu ne pojavljuje prije poruke m' .

Neka je p_k proces koji je poslao protokolarnu poruku $MSG(seq'' \oplus m'')$. Budući da je m' dio niza seq'' , proces p_k je morao CO-isporučiti poruku m' prije nego što je FIFO-broadcastao $MSG(seq'' \oplus m'')$.

Zbog FIFO svojstva, proces p_j je prije te protokolarne poruke morao FIFO-isporučiti sve ranije protokolarne poruke koje je poslao p_k .

Po pretpostavci, proces p_j nije CO-isporučio m prije m' . Stoga poruka m nije bila sadržana ni u jednoj ranijoj protokolarnoj poruci procesa p_k , niti se pojavljuje prije m' u nizu seq'' .

Iz toga slijedi da je i proces p_k CO-isporučio m' bez da je prethodno CO-isporučio m . No proces p_k je CO-isporučio m' prije procesa p_j , jer je to učinio prije slanja protokolarne poruke koju je kasnije primio p_j . Dakle, postoji trenutak $\tau' < \tau$ u kojem je neki proces CO-isporučio m' bez da je prethodno CO-isporučio m .

To je kontradikcija s definicijom trenutka τ kao prvog takvog trenutka.

Budući da oba slučaja vode na kontradikciju, tvrdnja C1 je dokazana.

Tvrdnja C2. Algoritam zadovoljava FIFO redoslijed isporuke aplikacijskih poruka.

Dokaz tvrdnje C2. Pretpostavimo da proces p_i CO-broadcasta poruku m prije poruke m' . Tada proces p_i FIFO-broadcasta protokolarnu poruku $MSG(seq \oplus m)$ prije protokolarne poruke $MSG(seq' \oplus m')$.

Neka je p_j bilo koji proces koji CO-isporuči m' . Budući da su obje protokolarne poruke poslone od istog procesa p_i , FIFO svojstvo osigurava da p_j FIFO-isporuča $MSG(seq \oplus m)$ prije $MSG(seq' \oplus m')$.

Prema tvrdnji C1, proces p_j ne može CO-isporučiti m' prije nego što je prethodno CO-isporučio m .

Stoga algoritam zadovoljava FIFO redoslijed za aplikacijske poruke.

Tvrdnja C3. Algoritam zadovoljava svojstvo lokalnog redoslijeda za aplikacijske poruke.

Dokaz tvrdnje C3. Neka je p_i proces koji CO-isporuči poruku m prije nego što CO-broadcasta poruku m' . Neka je p_j proces koji CO-isporuči poruku m' . Trebamo pokazati da p_j CO-isporuči m prije m' .

Neka je m'' prva poruka koju proces p_i CO-broadcasta nakon što je CO-isporučio poruku m . Moguće je da vrijedi $m'' = m'$.

Kada proces p_i CO-broadcasta poruku m'' , on FIFO-broadcasta protokolarnu poruku $MSG(seq'' \oplus m'')$ za neki niz seq'' .

Budući da je m'' prva poruka koju p_i šalje nakon što je CO-isporučio m , a algoritam sve CO-isporučine poruke od zadnjeg slanja sprema u *im_causal_past_i*, slijedi da se poruka m nalazi u nizu seq'' . Dakle, $m \in seq''$.

Prema tvrdnji C1, svaki proces koji CO-isporuči m'' mora prije toga CO-isporučiti m .

Ako je $m'' = m'$, onda odmah slijedi da p_j CO-isporuči m prije m' .

Ako je $m'' \neq m'$, tada je proces p_i CO-broadcastao m'' prije m' . Prema tvrdnji C2, koja kaže da algoritam zadovoljava FIFO redoslijed, svaki proces koji CO-isporuči m' mora prije toga CO-isporučiti m'' .

Budući da je svaki proces koji CO-isporuči m'' prethodno CO-isporučio m , slijedi da proces p_j CO-isporuči poruke redom

$$m \text{ prije } m'' \text{ prije } m'.$$

Dakle, p_j CO-isporuči m prije m' , pa algoritam zadovoljava lokalni redoslijed.

Prema tvrdnji C2, algoritam zadovoljava FIFO redoslijed isporuke aplikacijskih poruka. Prema tvrdnji C3, algoritam zadovoljava lokalni redoslijed.

Prema teoremu koji kaže da vrijedi

$$\text{kauzalni redoslijed} \equiv \text{FIFO redoslijed} + \text{lokalni redoslijed},$$

slijedi da algoritam zadovoljava svojstvo kauzalnog redoslijeda.

Budući da su dokazana svojstva valjanosti, integriteta, terminacije i kauzalnog redoslijeda, algoritam gradi CO-URB-broadcast komunikacijsku apstrakciju u svakom sustavu u kojem se može izgraditi FIFO-URB-broadcast. $\square$

5.2.6 Konstrukcija CO-broadcasta iz URB-broadcasta pomoću vektora

U ovom dijelu opisuje se konstrukcija CO-broadcast apstrakcije izravno nad URB-broadcast apstrakcijom. Za razliku od prethodne konstrukcije, koja se temeljila na FIFO-broadcastu, ova konstrukcija koristi vektorske satove kako bi se kauzalna prošlost poruke zapisala kompaktnije.

Osnovna ideja je da svaka poruka ne nosi cijeli niz poruka koje joj kauzalno prethode, nego nosi vektor brojeva. Taj vektor opisuje koliko je poruka od svakog procesa pošiljatelj već CO-isporučio prije slanja nove poruke.

Lokalni vektorski sat. Svaki proces p_i održava lokalni vektor $causal_past_i[1..n]$. Na početku vrijedi $causal_past_i = [0, 0, \dots, 0]$. Vrijednost $causal_past_i[k]$ označava broj poruka koje je proces p_k CO-broadcastao, a koje je proces p_i već CO-isporučio.

Drugim riječima, ako vrijedi $causal_past_i[k] = r$, to znači da je proces p_i već isporučio prvih r poruka koje je poslao proces p_k .

Budući da CO-broadcast uključuje i FIFO redoslijed, ovaj broj možemo shvatiti kao redni broj zadnje poruke procesa p_k koju je proces p_i dostavio.

Svaka aplikacijska poruka m nosi dodatno polje $m.causal_past[1..n]$. To polje je kopija lokalnog vektora procesa koji šalje poruku u trenutku slanja. Ako poruku m šalje proces p_i , tada se prije slanja postavlja

$$m.causal_past[1..n] \leftarrow causal_past_i[1..n].$$

Vrijednost $m.causal_past[k]$ označava broj poruka koje je proces p_k CO-broadcastao, a koje kauzalno prethode poruci m .

Formalno,

$$m.causal_past[k] = |\{m' : m' \text{ je CO-broadcastana od } p_k \text{ i } m' \rightarrow_M m\}|.$$

Dakle, vektor $m.causal_past$ opisuje kauzalnu prošlost poruke m .

Neka je poruka m URB-isporučena procesu p_i . Proces p_i smije CO-isporučiti poruku m tek ako je već lokalno CO-isporučio sve poruke koje kauzalno prethode poruci m .

To se provjerava pomoću uvjeta isporuke

$$DC_i(m) \equiv \forall k : causal_past_i[k] \geq m.causal_past[k].$$

Ovaj uvjet znači sljedeće: za svaki proces p_k , proces p_i mora već dostaviti barem onoliko poruka od procesa p_k koliko ih poruka m ima u svojoj kauzalnoj prošlosti.

Ako je uvjet $DC_i(m)$ zadovoljen, tada proces p_i smije CO-isporučiti poruku m .

Ako uvjet nije zadovoljen, poruka m ne smije još biti isporučena, nego se sprema u skup čekanja msg_set_i .

Algoritam.

operation $CO_broadcast(m)$ **is**

- (1) $done_i \leftarrow false$;
- (2) $m.causal_past[1..n] \leftarrow causal_past_i[1..n]$;
- (3) $m.sender \leftarrow i$;
- (4) $URB_broadcast\ MSG(m)$;
- (5) $wait(done_i)$.

when $MSG(m)$ **is urb-delivered do**

- (6) **if** $DC_i(m)$ **then**
- (7) $CO_deliver(m)$;
- (8) **let** $j = m.sender$;
- (9) $causal_past_i[j] \leftarrow m.causal_past_i[j] + 1$;
- (10) $done_i \leftarrow (m.sender = i)$;
- (11) **while** $\exists m' \in msg_set_i : DC_i(m')$ **do**
- (12) $CO_deliver(m')$;
- (13) **let** $j = m'.sender$;
- (14) $causal_past_i[j] \leftarrow m'.causal_past_i[j] + 1$;
- (15) $msg_set_i \leftarrow msg_set_i \setminus \{m'\}$;
- (16) **end while**
- (17) **else** $msg_set_i \leftarrow msg_set_i \cup \{m\}$;
- (18) **end if.**

Objašnjenje slanja poruke. Kada proces p_i želi CO-broadcastati poruku m , on najprije postavlja $done_i \leftarrow false$. Zatim u poruku m kopira svoj trenutni vektor kauzalne prošlosti:

$$m.causal_past[1..n] \leftarrow causal_past_i[1..n].$$

Nakon toga u poruku zapisuje svoj identitet: $m.sender \leftarrow i$. Zatim poruku šalje pomoću URB-broadcasta: $URB_broadcast\ MSG(m)$. Na kraju proces čeka dok lokalno ne CO-isporuči vlastitu poruku. To je označeno naredbom $wait(done_i)$. Varijabla $done_i$ služi za to da proces ne pošalje sljedeću poruku prije nego što je njegova prethodna poruka pravilno uključena u njegov lokalni vektor $causal_past_i$.

Objašnjenje primanja poruke. Kada proces p_i URB-isporuči poruku m , najprije provjerava vrijedi li uvjet $DC_i(m)$.

Ako uvjet vrijedi, to znači da je proces p_i već dostavio sve poruke koje kauzalno prethode poruci m . Zato smije izvršiti $CO_deliver(m)$.

Nakon toga se ažurira lokalni vektor. Ako je poruku m poslao proces p_j , tada se postavlja

$$causal_past_i[j] \leftarrow m.causal_past_i[j] + 1.$$

Ovo znači da je proces p_i sada isporučio još jednu poruku od procesa p_j .

Ako je poruka m vlastita poruka procesa p_i , tada se postavlja $done_i \leftarrow true$. Time se omogućuje nastavak operacije $CO_broadcast(m)$.

Nakon svake CO-isporuke proces provjerava postoje li u skupu čekanja msg_set_i neke poruke koje sada zadovoljavaju uvjet isporuke. Ako postoje, one se redom CO-isporučuju, uklanjaju iz skupa čekanja i ažurira se vektor $causal_past_i$.

Ako uvjet $DC_i(m)$ ne vrijedi, tada poruka još ne smije biti CO-isporučena. Zato se sprema u skup čekanja:

$$msg_set_i \leftarrow msg_set_i \cup \{m\}.$$

Primjer. Pretpostavimo da imamo tri procesa p_1, p_2, p_3 . Na početku svi imaju vektor $[0, 0, 0]$.

Neka proces p_1 pošalje poruku m_1 . Budući da prije toga nije isporučio ni jednu poruku, poruka m_1 nosi vektor $m_1.causal_past = [0, 0, 0]$. Neka proces p_2 također pošalje poruku m_2 , neovisno o poruci m_1 . Tada i poruka m_2 nosi vektor $m_2.causal_past = [0, 0, 0]$.

Poruke m_1 i m_2 nisu kauzalno povezane, pa ih različiti procesi smiju CO-isporučiti različitim redoslijedom.

Sada pretpostavimo da proces p_3 najprije CO-isporuči poruku m_1 , a zatim CO-broadcasta poruku m_3 . Budući da je p_3 prije slanja poruke m_3 već dostavio jednu poruku od procesa p_1 , poruka m_3 nosi vektor $m_3.causal_past = [1, 0, 0]$. To znači da poruci m_3 kauzalno prethodi prva poruka procesa p_1 , odnosno poruka m_1 .

Ako neki proces, primjerice p_2 , URB-isporuči poruku m_3 prije poruke m_1 , tada njegov lokalni vektor može biti $causal_past_2 = [0, 0, 0]$. Uvjet isporuke za m_3 glasi

$$causal_past_2[1] \geq m_3.causal_past[1].$$

Odnosno

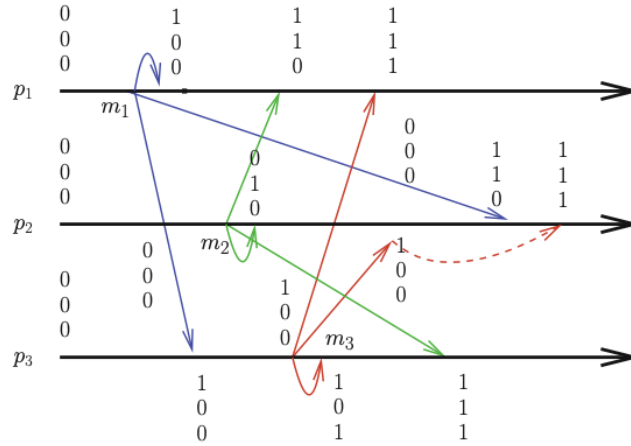
$$0 \geq 1.$$

To nije istina, pa proces p_2 ne smije CO-isporučiti poruku m_3 . Umjesto toga, poruku m_3 sprema u skup čekanja msg_set_2 .

Kada proces p_2 kasnije CO-isporuči poruku m_1 , njegov lokalni vektor postaje $causal_past_2 = [1, 0, 0]$. Sada je uvjet isporuke za poruku m_3 zadovoljen:

$$1 \geq 1.$$

Zato proces p_2 sada smije CO-isporučiti poruku m_3 .



Slika 7: Enter Caption

Zašto algoritam osigurava kauzalni redoslijed? Svaka poruka nosi vektor koji opisuje njezinu kauzalnu prošlost. Ako poruka m u svom vektoru ima $m.causal_past[k] = r$, to znači da prije poruke m mora biti dostavljeno prvih r poruka koje je poslao proces p_k .

Proces p_i smije CO-isporučiti poruku m samo ako njegov lokalni vektor zadovoljava

$$causal_past_i[k] \geq m.causal_past[k]$$

za svaki k .

Dakle, proces p_i mora već imati dostavljene sve poruke koje su zapisane u kauzalnoj prošlosti poruke m . Zato se poruka nikada ne može CO-isporučiti prije poruka koje joj kauzalno prethode.

Zaključak. Ova konstrukcija CO-broadcasta koristi URB-broadcast za pouzdanu isporuku poruka, a vektorske satove za očuvanje kauzalnog redoslijeda.

Za razliku od prethodnih konstrukcija, poruke ne nose cijelu kauzalnu prošlost kao niz poruka, nego nose samo vektor brojeva. Zato je ova konstrukcija praktičnija i učinkovitija.

Glavni uvjet isporuke je

$$DC_i(m) \equiv \forall k : \text{causal_past}_i[k] \geq m.\text{causal_past}[k].$$

On osigurava da proces p_i može CO-isporučiti poruku m tek kada je već dostavio sve poruke koje kauzalno prethode poruci m .

Lema 1. *Neka su m i m' dvije različite aplikacijske poruke. Ako vrijedi $m \rightarrow_M m'$, tada vrijedi*

$$\begin{aligned} &(\forall k : m.\text{causal_past}[k] \leq m'.\text{causal_past}[k]) \\ &\wedge (\exists k : m.\text{causal_past}[k] < m'.\text{causal_past}[k]). \end{aligned}$$

Dokaz. Najprije promatramo slučaj u kojem su poruke m i m' CO-broadcastane od istog procesa p_i .

Zbog načina na koji se koristi Booleova varijabla $done_i$, proces p_i ne može započeti sljedeći CO-broadcast prije nego što je njegova prethodna poruka lokalno CO-isporučena. Također, svaki put kada proces p_i CO-isporuči poruku koju je sam CO-broadcastao, povećava vrijednost svoje komponente vektora $\text{causal_past}_i[i]$ za jedan.

Zato su svaka dva uzastopna poziva operacije *CO_broadcast* od procesa p_i razdvojena ažuriranjem

$$\text{causal_past}_i[i] \leftarrow \text{causal_past}_i[i] + 1.$$

Budući da je poruka m CO-broadcastana prije poruke m' , slijedi

$$m.\text{causal_past}[i] < m'.\text{causal_past}[i].$$

Za sve ostale komponente $k \neq i$ vrijedi da se vrijednosti $causal_past_i[k]$ tijekom izvođenja algoritma nikada ne smanjuju. One se mogu samo povećavati kada proces p_i CO-isporuči nove poruke.

Zato vrijedi

$$\forall k : m.causal_past[k] \leq m'.causal_past[k].$$

Budući da je za komponentu i nejednakost stroga, dobivamo

$$\begin{aligned} & (\forall k : m.causal_past[k] \leq m'.causal_past[k]) \\ & \wedge (\exists k : m.causal_past[k] < m'.causal_past[k]). \end{aligned}$$

Time je lema dokazana za slučaj kada su m i m' CO-broadcastane od istog procesa.

Sada promatramo slučaj u kojem su poruke m i m' CO-broadcastane od različitih procesa.

Budući da vrijedi $m \rightarrow_M m'$, postoji konačan lanac poruka

$$m = m_0 \rightarrow_M m_1 \rightarrow_M \cdots \rightarrow_M m_z = m',$$

takav da za svaku poruku m_x , gdje je $1 \leq x \leq z$, proces koji je CO-broadcastao poruku m_x prethodno je CO-isporučio poruku m_{x-1} .

Tvrdimo da vrijedi

$$\begin{aligned} & (\forall k : m.causal_past[k] \leq m_1.causal_past[k]) \\ & \wedge (\exists k : m.causal_past[k] < m_1.causal_past[k]). \end{aligned}$$

Ako to pokažemo za prvi korak lanca, tada tvrdnja leme slijedi jednostavnom indukcijom po duljini lanca poruka.

Neka je p_i proces koji je CO-broadcastao poruku m , a neka je p_j , gdje je $i \neq j$, proces koji je CO-isporučio poruku m prije nego što je CO-broadcastao poruku m_1 .

Iz definicije relacije $m \rightarrow_M m_1$, iz činjenice da je p_j CO-isporučio poruku m , te iz uvjeta isporuke $DC_j(m)$, slijedi da neposredno nakon CO-isporuke poruke m u procesu p_j vrijedi

$$\forall k : m.causal_past[k] \leq causal_past_j[k].$$

S druge strane, kada je proces p_j CO-isporučio poruku m , izvršio je ažuriranje komponente koja odgovara pošiljatelju poruke m , odnosno procesu p_i :

$$causal_past_j[i] \leftarrow causal_past_j[i] + 1.$$

Zato nakon CO-isporuke poruke m vrijedi

$$m.causal_past[i] < m.causal_past[i] + 1 = causal_past_j[i].$$

Općenito, dobivamo

$$\begin{aligned} & (\forall k : m.causal_past[k] \leq causal_past_j[k]) \\ & \wedge (\exists k : m.causal_past[k] < causal_past_j[k]). \end{aligned}$$

Kada proces p_j nakon toga CO-broadcasta poruku m_1 , on u poruku kopira svoj trenutni vektor kauzalne prošlosti:

$$m_1.causal_past[1..n] = causal_past_j[1..n].$$

Budući da se to događa nakon CO-isporuke poruke m , slijedi

$$\begin{aligned} & (\forall k : m.causal_past[k] \leq m_1.causal_past[k]) \\ & \wedge (\exists k : m.causal_past[k] < m_1.causal_past[k]). \end{aligned}$$

Time je tvrdnja za prvi korak lanca dokazana.

Primjenom iste argumentacije na svaki sljedeći korak lanca

$$m_0 \rightarrow_M m_1 \rightarrow_M \cdots \rightarrow_M m_z$$

dobivamo da je vektor kauzalne prošlosti komponentno nepadajući duž cijelog lanca, te da je u barem jednoj komponenti strogo veći. Budući da je $m_0 = m$ i $m_z = m'$, slijedi

$$\begin{aligned} & (\forall k : m.causal_past[k] \leq m'.causal_past[k]) \\ & \wedge (\exists k : m.causal_past[k] < m'.causal_past[k]). \end{aligned}$$

Time je lema dokazana. □

Teorem 3. *Opisan algoritam gradi komunikacijsku apstrakciju CO-URB-broadcast u svakom sustavu u kojem se može izgraditi URB-broadcast.*

Dokaz. Dokazujemo da algoritam zadovoljava svojstva CO-URB-broadcast apstrakcije: valjanost, integritet, terminaciju i kauzalni redoslijed isporuke.

Valjanost i integritet. Svojstvo valjanosti slijedi izravno iz svojstva valjanosti temeljne URB-broadcast apstrakcije. Algoritam ne stvara nove aplikacijske poruke proizvoljno, nego samo šalje poruke koje su stvarno CO-broadcastane.

Svojstvo integriteta URB-broadcasta jamči da za svaku aplikacijsku poruku m koja je CO-broadcastana proces p_i može URB-iskoručiti najviše jednu protokolarnu poruku $MSG(m)$. Ako je uvjet isporuke $DC_i(m)$ zadovoljen, poruka m se odmah CO-iskoruči. Ako uvjet nije zadovoljen, poruka se sprema u skup čekanja msg_set_i . Kada kasnije bude CO-iskoručena, uklanja se iz tog skupa.

Iz toga slijedi da nijedna poruka m ne može biti CO-iskoručena više od jednom kod istog procesa.

Time su zadovoljena svojstva valjanosti i integriteta.

Terminacija. Svojstvo terminacije temeljne URB-broadcast apstrakcije jamči sljedeće:

1. ako ispravan proces CO-broadcasta poruku m , tada on URB-broadcasta protokolarnu poruku $MSG(m)$;
2. ako bilo koji proces URB-iskoruči $MSG(m)$, tada će svaki ispravan proces eventualno URB-iskoručiti $MSG(m)$.

Iz toga slijedi da će svaki ispravan proces p_i za poruku m eventualno napraviti jedno od sljedećeg: $CO_deliver(m)$ ili $msg_set_i \leftarrow msg_set_i \cup \{m\}$.

Dakle, da bismo dokazali terminaciju CO-broadcasta, dovoljno je pokazati da svaki ispravan proces eventualno CO-iskoruči sve poruke koje su spremljene u njegov skup čekanja msg_set_i .

Pretpostavimo suprotno, tj. da neke poruke zauvijek ostaju u skupu čekanja nekog ispravnog procesa p_i . Označimo skup takvih poruka s $blocked_i$.

Poruke u tom skupu uredimo prema leksikografskom poretку njihovih vektora $m.causal_past$. Za dva vektora

$$v = [a, b, c] \quad \text{ i } \quad v' = [a', b', c']$$

pišemo $v <_{lex} v'$ ako vrijedi

$$(a < a') \vee (a = a' \wedge b < b') \vee (a = a' \wedge b = b' \wedge c < c').$$

Neka je m prva poruka u skupu $blocked_i$ prema leksikografskom poretку. Neka je p_x proces koji je izvršio $CO_broadcast(m)$.

Budući da poruka m zauvijek ostaje u msg_set_i , uvjet isporuke $DC_i(m)$ zauvijek ostaje neistinit. Zato postoji barem jedan indeks k takav da

$$0 \leq causal_past_i[k] < m.causal_past[k].$$

Neka je

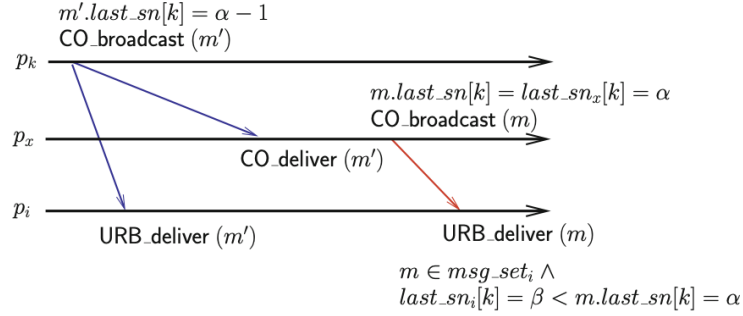
$$m.causal_past[k] = \alpha.$$

Budući da je α konstanta, a vrijednost $causal_past_i[k]$ se tijekom izvođenja samo povećava, postoji konačna posljednja vrijednost te komponente. Označimo je s $\beta < \alpha$. Budući da vrijedi

$$causal_past_x[k] = m.causal_past[k] = \alpha \geq 1,$$

proces p_x je prije slanja poruke m morao CO-isporučiti neku aplikacijsku poruku m' koju je CO-broadcastao proces p_k , takvu da vrijedi

$$m'.causal_past[k] = \alpha - 1.$$



Slika 8: Enter Caption

Budući da je proces p_x CO-isporučio poruku m' , on je prethodno morao URB-isporučiti $MSG(m')$. Iz svojstva terminacije URB-broadcasta slijedi da će svaki ispravan proces, pa tako i proces p_i , eventualno URB-isporučiti $MSG(m')$.

Kada proces p_i URB-isporuči poruku m' , postoje dva slučaja.

Slučaj 1. Uvjet isporuke $DC_i(m')$ je neistinit i ostaje neistinit zauvijek.

Budući da vrijedi $m' \rightarrow_M m$, prema Lemi 1 vrijedi

$$m'.causal_past <_{lex} m.causal_past.$$

To znači da bi poruka m' trebala biti prije poruke m u leksikografskom poretku skupa $blocked_i$. To je kontradikcija s izborom poruke m kao prve poruke u skupu $blocked_i$.

Slučaj 2. Poruka m' je eventualno CO-isporučena od procesa p_i .

Tada se komponenta $causal_past_i[k]$ povećava za jedan, pa postaje jednaka $\beta + 1$. To je kontradikcija s pretpostavkom da je β posljednja vrijednost koju je komponenta $causal_past_i[k]$ poprimila.

U oba slučaja dobivamo kontradikciju. Zato nijedna poruka ne može zauvijek ostati u skupu čekanja msg_set_i . Time je terminacija CO-broadcasta dokazana.

Kauzalni redoslijed isporuke. Preostaje dokazati da algoritam zadovoljava svojstvo kauzalnog redoslijeda isporuke.

Promotrimo poruku m koju je CO-broadcastao proces p_j . Budući da su vektori $causal_past_j[1..n]$ inicijalizirani na $[0, \dots, 0]$ i ažuriraju se pri svakoj CO-isporuci poruke, vektor $m.causal_past[1..n]$ točno opisuje kauzalnu prošlost poruke m .

Neka je M_1 skup svih poruka m' takvih da vrijedi $m' \rightarrow_M m$.

Tada vektor $m.causal_past[1..n]$ kodira upravo skup M_1 , odnosno sve poruke koje kauzalno prethode poruci m .

Sada promatramo proces p_i koji je URB-isporučio $MSG(m)$.

Neka je trenutak u kojem uvjet isporuke $DC_i(m)$ postaje zadovoljen. U tom trenutku lokalni vektor $causal_past_i[1..n]$ opisuje trenutnu kauzalnu prošlost procesa p_i , odnosno skup M_2 svih poruka koje je proces p_i već CO-isporučio.

Uvjet isporuke $DC_i(m)$ definiran je kao

$$DC_i(m) \equiv \forall k : causal_past_i[k] \geq m.causal_past[k].$$

Ovaj uvjet upravo kaže da je proces p_i već CO-isporučio sve poruke koje se nalaze u kauzalnoj prošlosti poruke m . Drugim riječima,

$$M_1 \subseteq M_2.$$

Zato proces p_i može CO-isporučiti poruku m samo ako je već CO-isporučio sve poruke koje kauzalno prethode poruci m .

Time je dokazano svojstvo kauzalnog redoslijeda.

Zaključak. Pokazali smo da algoritam zadovoljava valjanost, integritet, terminaciju i kauzalni redoslijed isporuke. Stoga algoritam gradi CO-URB-broadcast komunikacijsku apstrakciju u svakom sustavu u kojem se može izgraditi URB-broadcast. $\square$

6 Zaključak

Pokazali smo kako URB algoritam te njegova FIFO i CO proširenja osiguravaju da svi korektni procesi u distribuiranom asinkronom sustavu naposljetku sadrže isti skup primljenih poruka. Također smo pokazali kako FIFO-URB proširenje dodatno jamči isporučivanje poruka koje neki proces pošalje onim redoslijedom kojim su poslone. CO-URB proširenje osigurava proširena svojstva od FIFO-URB-a, uz dodatne slučajeve kauzalnog prethođenja, na taj način da se poruke isporučuju prema svojem kauzalnom poretku (u tom smislu je FIFO-URB poseban slučaj CO-URB-a). Svako proširenje osigurava jača svojstva bez dodatnih pretpostavki na sam distribuirani sustav. Prikazane su konstrukcije algoritama u pseudokodu te su demonstrirane implementacije na testnim primjerima.

7 Analiza i ograničenja algoritma

U osnovnom modelu asinkronog sustava s pouzdanim komunikacijskim kanalima, URB se može implementirati jednostavnom strategijom “prvo proslijedi, a zatim isporuči”. Kada proces prvi put primi poruku, on je najprije prosljeđuje svim ostalim procesima, a tek zatim je isporučuje lokalnoj aplikaciji.

Iako je ova strategija konceptualno jednostavna, postoje važna teorijska ograničenja. U asinkronim sustavima s mogućnošću crash failure kvarova, implementacija URB-a nije moguća ako je broj mogućih kvarova procesa veći ili jednak polovici ukupnog broja procesa ($t \geq n/2$), pod pretpostavkom nepouzdanih, ali fer kanala.

Kako bi se ova ograničenja zaobišla, sustavi se često proširuju dodatnim mehanizmima poput detektora kvarova (npr. klasa Θ), koji omogućuju procesima da steknu dodatne informacije o tome koji su procesi vjerojatno ispravni. Time se povećava mogućnost implementacije pouzdanih broadcast protokola u realnim distribuiranim sustavima.

Literatura

- [1] Michel Raynal. Fault-tolerant message-passing distributed systems. In *An Algorithmic Approach*, chapter 2. Springer, 2018.